

Disaster Management Simulation Report

Course: Object-Oriented Programming

Student: Pranav Varanganti

Professor: Salvatore Distefano

University: University of Messina

Date: 30 October 2025

Table of Contents

1. Introduction
 2. Project Aim
 3. Some main Events
 - 3.1 Disaster Occur Event
 - 3.2 Single Disaster Event
 - 3.3 Rescue operation flow (RescueArrival and RescueOperationEnd Events)
 - 3.4 Ambulance operation flow (AmbulanceArrival and AmbulanceOperationEnd Events)
 - 3.5 Hospital Treatment Event
 - 3.6 Weather Change Event
 4. Object-Oriented Programming Concepts Used
 - 4.1 Encapsulation
 - 4.2 Information Hiding
 - 4.3 Inheritance
 - 4.4 Composition
 - 4.5 Abstraction
 - 4.6 Subtyping (and Multityping with Interfaces)
 - 4.7 Polymorphism
 - 4.7.1 Polymorphism by Inclusion (Inclusion Polymorphism)
 - 4.7.2 Parametric Polymorphism
 - 4.7.3 Coercion Polymorphism
 - 4.7.4 Overloading
 - 4.7.5 Method Overriding / Runtime Polymorphism
 - 4.8 Exception Handling
 - 4.9 Extensibility
 5. Simulation Logic
 6. Extensibility of project
 7. Conclusion
-

1. Introduction

This report describes the implementation of a Disaster Management Simulation using object-oriented programming principles. The project simulates a city where disasters occur, rescue operations are conducted, hospitals admit patients, and resources are managed. Each OOP concept used in the project is described with code examples and motivation.

This report presents the development of a **Disaster Management Simulation** that leverages the power of **object-oriented programming (OOP)** to model complex real-world scenarios. The simulation captures the dynamics of a city facing various disasters, coordinating rescue operations, hospital admissions, and efficient resource management. Each core OOP concept—ranging from inheritance and polymorphism to encapsulation and exception handling—is carefully applied to enhance modularity, maintainability, and realism. Throughout the report, code examples are provided to illustrate the implementation choices and demonstrate how OOP principles drive the simulation's design and functionality.

2. Project Aim

The goal of this project is to simulate a disaster management system within a single city, where various entities such as zones, hospitals and various resource units such as rescue units, ambulance units and food supply teams interact to respond to emergencies. The simulation models disasters like floods, earthquakes, fires, and thunderstorms, coordinating rescue operations, hospital admissions, and resource allocation to ensure realistic outcomes.

Some key features:

- Disasters occur dynamically at scheduled times, impacting different zones with varying severity (depends on weather).
- Rescue units, volunteer teams, ambulance units, and police respond to disasters, saving lives and managing casualties.
- Hospitals admit injured individuals, simulate births, simulate deaths and track discharges with capacity management included.
- Food supply and other resources are monitored, with mechanisms to prevent shortages and handle exceptions.
- A discrete-event simulation ensures that all actions are executed in chronological order, maintaining realism.
- The Observer pattern is applied to the Simulation Manager to notify all relevant events about weather changes.
- The entire system is designed using OOP principles, promoting modularity, maintainability, and extensibility.

3. Some main Events:

3.1. DisasterOccur Event:

This event triggers once per day. It decides how many disasters will happen that day and in which zones they will occur. It randomly selects the disaster type (e.g., Flood, Fire Accident, Earthquake, Thunderstorm) and schedules multiple `SingleDisasterEvent` instances through Event calendar.

3.2. SingleDisaster Event:

Represents the detailed setup for one specific disaster. It creates a Disaster object, initializes casualties. It then schedules follow-up events such as rescue and medical responses.

3.3. Rescue operation flow (RescueArrival and RescueOperationEnd Events):

One hour after a disaster is triggered by the **SingleDisasterEvent**, the **RescueTeamArrivalEvent** is scheduled. In this phase, the system assigns an available Rescue Unit to the affected zone. If no rescue units are currently available, a Volunteer Team is dynamically created to ensure that rescue operations continue without delay.

After one additional hour, the **RescueOperationEndEvent** is executed to simulate the outcome of the rescue mission. During this event, the system calculates the number of people rescued, injured, and dead based on the total number of trapped individuals in the disaster. These outcomes directly affect the city's population statistics and hospital admissions. Once the operation concludes, if the rescue was performed by a volunteer team, that team is dismantled to reflect their temporary deployment.

3.4. Ambulance operation flow (AmbulanceArrival and AmbulanceOperationEnd Events):

Two hours after a disaster is triggered by the **SingleDisasterEvent**, the **AmbulanceArrivalEvent** is scheduled. During this phase, an available Ambulance Unit is assigned to the affected zone to transport injured individuals to hospitals. If no ambulance units are available, a Volunteer Team is temporarily assigned to ensure timely medical evacuation.

One hour after the ambulance arrives, the **AmbulanceOperationEndEvent** is executed. This event handles the admission of injured patients from the disaster into the appropriate hospitals. The hospital occupancy and patient statistics are updated accordingly. Once the operation concludes, if a volunteer team performed the ambulance duties, the team is dismantled, reflecting their temporary deployment.

3.5. Hospital Treatment Event: Simulates births, deaths (in hospital) and discharges of patients.

3.6. Weather Change Event: Changes weather and notifies all the observers.

4. Object-Oriented Programming Concepts Used

4.1 Encapsulation

Definition: Encapsulation is the object-oriented principle of **binding data(attributes) and the methods that operate on that data inside a single class**, while providing a controlled interface for interaction.

How I applied: In my project, each class encapsulates its own attributes and behaviors — for example, the Hospital class contains fields such as capacity, occupiedBeds, and methods like `admitPatient()` or `dischargePatient()`. External components cannot directly modify the hospital's internal state; instead,

they must use these public methods. This ensures that the state of a class is controlled and consistent, preventing accidental or invalid updates from other parts of the simulation.

Where it appears: In every core class such as Hospital, Disaster, Event, and SimulationManager classes.

Code Snippet:

```
public abstract class Disaster {
    private String name;
    private int severity;        // 1-5
    private Zone zone;
    // Results of the disaster
    private int injured;
    private int deaths;
    private int trapped;
    private int rescued;
    private int injuredToAdmit;

    public String getName() {return name;}
    public int getSeverity() {return severity;}
    public Zone getZone() {return zone;}
    public int getInjured() {return injured;}
    public int getDeaths() {return deaths;}
    public int getTrapped() {return trapped;}
    public int getRescued() {return rescued;}
    public int getInjuredToAdmit() {return injuredToAdmit;}

    public void setInjuredToAdmit(int injuredToAdmit) {this.injuredToAdmit = injuredToAdmit;}
    public void setRescued(int rescued) {this.rescued = rescued;}
    public void setInjured(int injured) {this.injured = injured;}
    public void setDeaths(int deaths) {this.deaths = deaths;}
    public void setTrapped(int trapped) {this.trapped = trapped;}
    public void setSeverity(int severity) {this.severity = severity;}
    public void setZone(Zone zone) {this.zone = zone;}
}
```

Explanation: In this class we encapsulated the data(injured, deaths, trapped and etc.) and methods (public getters and setters) are binded. So, external components cannot directly modify the Disaster's internal state; instead, they must use these public methods. This ensures controlled access, prevents inconsistent states, and keeps each object responsible for managing its own data. Encapsulation therefore strengthens modularity and maintains a clean separation of responsibilities within the system.

4.2 Information Hiding

Definition: Information hiding is the practice of hiding implementation details and exposing only what is necessary through well-defined interfaces.

How I applied: Information hiding was achieved by three steps:

- 1) Declaring all the instance variables as private.

```
private String name;
private int severity;
private Zone zone;
// Results of the disaster
private int injured;
private int deaths;
private int trapped;
private int rescued;
private int injuredToAdmit;
```

- 2) Declaring class level variables (shared by all instances) as private static and instance level variables (unique to each instance) as private.

```
private int hour;  
private int priority;  
  
private static int totalPopulation = 800;  
private static int totalFoodPercentage = 100;
```

- 3) Declaring constants as private final.

```
private final int[] ranges = {10, 30, 0, 5, 0, 25};  
//[injuredMin, injuredMax, deathMin, deathMax, trappedMin, trappedMax]
```

Where it appears: In every class such as Hospital, Disaster, Event and Simulation Manager classes.

Explanation: All variables in the project are declared as private, ensuring that direct access from external classes is restricted. This design enforces data hiding and maintains the integrity of each object's internal state. The internal state of totalPopulation and totalFoodPercentage is hidden, and access is only allowed via getter/setter methods, allowing safe and controlled updates. Additionally, constant values such as the ranges used to generate disaster attributes—like injuries, deaths, and trapped individuals—are declared as final, preventing accidental modification and ensuring consistency throughout the simulation lifecycle.

4.3 Inheritance

Definition: Inheritance allows one class (called a subclass or child class) to acquire the properties and behaviors (fields and methods) of another class (called a superclass or parent class).

How I applied: Inheritance is used throughout the project to create hierarchies of related entities. For instance, all disaster types like FloodDisaster, EarthquakeDisaster, and FireAccidentDisaster extend the abstract Disaster class.

Where it appears: City, Zone and Hospital extends Entity, RescueUnit, AmbulanceUnit, PoliceUnit etc. extends Resources, Earthquake, Floods, Fire accident etc. extends Disasters. Also, some more classes.

Code Snippets:

```
public class City extends Entity {
```

```
public class Zone extends Entity{
```

```
public class EarthquakeDisaster extends Disaster {
```

```
public class FloodDisaster extends Disaster {
```

```
public class AmbulanceUnit extends Resource {
```

```
public class RescueUnit extends Resource {
```

Explanation: Classes such as City, Zone, and Hospital inherit shared attributes and behaviors from the Entity superclass, while each provides its own specialized implementation for unique functionalities. Similarly, all subclasses in the project extend their respective super classes to reuse common logic, ensuring consistency, reducing redundancy, and allowing each subclass to introduce specific behavior relevant to its role within the simulation.

4.4 Composition

Definition: Composition is when a class is built using references to other classes, establishing a “**has-a**” relationship rather than an “is-a” relationship. It allows complex objects to be constructed from simpler, reusable components, promoting modularity and flexibility.

How I applied: In my project, every Event class contains Zone, Disaster, EventCalendar, TimeManager and SimulationManager.

Where it appears: Single Disaster Event has zone, Event calendar, Time manager and SimulationManager. SimulationManager has lists of hospitals, zones, and resources.

Code Snippets:

```
public class SingleDisasterEvent extends Event {
    private Zone zone;
    private EventCalendar calendar;
    private TimeManager tm;
    private SimulationManager sm;
    private int severity;
    private String type;
```

Explanation: The SingleDisasterEvent class demonstrates **composition** because it contains and manages objects like Zone, EventCalendar, TimeManager and SimulationManager. These components together define the state and behavior of a disaster scenario. Once the event is processed, the associations between these objects are no longer relevant, showing that their lifecycle depends on the event itself. This approach keeps disaster-related logic localized within the event and improves modularity, readability, and maintainability of the system.

```
public class SimulationManager { //implements WeatherObserver
    private final EntityRegistry<Hospital> hospitalRegistry = new EntityRegistry<>();
    private final EntityRegistry<Zone> zoneRegistry = new EntityRegistry<>();

    private final ResourceRegistry<RescueUnit> rescueRegistry = new ResourceRegistry<>();
    private final ResourceRegistry<AmbulanceUnit> ambulanceRegistry = new ResourceRegistry<>();
    private final ResourceRegistry<FoodSupplyTeam> foodRegistry = new ResourceRegistry<>();
    private final ResourceRegistry<PoliceUnit> policeRegistry = new ResourceRegistry<>();
    private final ResourceRegistry<VolunteerTeam> volunteerRegistry = new ResourceRegistry<>();
```

Explanation: The SimulationManager demonstrates **composition** by combining several registry objects (EntityRegistry, ResourceRegistry) that manage hospitals, zones, and resources. These components are integral parts of the manager and cannot exist independently. Declaring them as final ensures they remain constant throughout the simulation, reinforcing that the SimulationManager is composed of and depends on these subsystems to function effectively.

4.5 Abstraction

Definition: Abstraction is an object-oriented programming principle that focuses on representing only the essential features of an object while hiding complex background details or implementation logic. It simplifies system complexity by exposing only the necessary behavior through well-defined interfaces or abstract classes. In other words, abstraction lets programmers interact with objects at a high level without worrying about how the functionality is internally implemented.

How I applied: Abstraction is applied in the project through the use of the abstract classes like Event, Entity and interfaces such as EndOfTheDay and WeatherObserver.

Where it appears: Abstract classes Event, Entity, Disaster, Resource. Interfaces EndOfTheDay, WeatherObserver

Code Snippets:

```
public abstract class Event implements Comparable<Event>, WeatherObserver, EndOfTheDay {
    private int hour;
    private int priority;

    private static int totalPopulation = 800;
    private static int totalFoodPercentage = 100;
    private static int totalBirths=0;
    private static int totalDeaths=0;
    private static int totalInjured=0;
    private static int totalRescued=0;
    private static int totalDisasters=0;

    private static int dailyBirths=0;
    private static int dailyDeaths=0;
    private static int dailyInjured=0;
    private static int dailyRescued=0;
    private static int dailyDisasters=0;
    private static String latestWeather = "clear";

    public Event(int hour, int priority) {
        this.hour = hour;
        this.priority = priority;
    }

    public abstract void process();
}
```

Explanation: The Event class defines the structure and behavior of all simulation events (e.g., disasters, rescues, ambulance arrivals) using the abstract method process(). Each specific event type (like SingleDisasterEvent, RescueTeamArrivalEvent, or EndOfTheDayEvent) provides its own implementation of this method, hiding the internal complexity from other parts of the program.

```
public abstract class Disaster {
    private String name;
    private int severity; // 1-5
    private Zone zone;
    // Results of the disaster
    private int injured;
    private int deaths;
    private int trapped;
    private int rescued;
    private int injuredToAdmit;

    // Each subclass provides its base min/max ranges
    protected abstract int[] getBaseRanges();
}
```

Explanation: The Disaster class defines the common structure and core behavior shared by all disaster types—such as storing severity, affected zone, and casualty statistics. However, it hides the internal logic for how these values are generated and requires subclasses to implement the abstract method getBaseRanges(). This allows each specific disaster type to define its own behavior without exposing unnecessary details. The rest of the system can interact with any disaster through this abstract interface, making the design flexible and extensible.

4.6 Subtyping (and Multityping with Interfaces)

Definition: Subtyping refers to the ability of an object of a subclass to be used wherever its superclass or interface type is expected. In other words, a subtype can seamlessly substitute for its supertype, as it adheres to the same external interface. This promotes flexibility, extensibility, and code reusability, allowing systems to handle objects based on shared behavior rather than specific class implementations.

How I applied: I demonstrated subtyping and multityping with interfaces in the main class by showing how can I do a demo disaster event by subtyping and multityping with interfaces.

Where it appears: Disaster class, Hospital class and Event class.

Code Snippets:

```
public abstract class Disaster implements SeverityCalculator{
```

```
public class Hospital extends Entity implements EndOfTheDay {
```

```
public abstract class Event implements Comparable<Event>, WeatherObserver,EndOfTheDay {
```

```
System.out.println("\n---- Subtyping and Multityping Demo in Disaster Management System ----");
Zone demozone = new Zone("demoZone");
Hospital demoHospital = new Hospital(demozone,sm);
Resource demoRescue = new RescueUnit("Demo Rescue");
Resource demoAmbulance = new AmbulanceUnit("Demo Ambulance");
Resource demoPolice = new PoliceUnit("DemoPolice");

// subtyping and multityping through abstract class Event and Interfaces EndOfTheDay and WeatherObserver
EndOfTheDayEvent demoevent = new EndOfTheDayEvent(0,null,null);
Event g1 = demoevent; // Access via abstract class
EndOfTheDay IEnd = demoevent; // Access via Interface
WeatherObserver Iweather = demoevent; // Access via Interface
Iweather.onWeatherUpdate("Rain");
System.out.println("Weather updated to "+ Event.getLatestWeather());

// subtyping and multityping through abstract class Disaster and Interface SeverityCalculator
FloodDisaster demoDisaster = new FloodDisaster();
Disaster disaster = demoDisaster; // Access via abstract class
SeverityCalculator severity = demoDisaster; // Access via Interface
disaster.setZone(demozone);
disaster.setSeverity(severity.determineSeverity("Rain")); // severity determined through the interface SeverityCalculator
disaster.initializeCasualties();
Event.addDailyDisasters(1);
demozone.setDisasterOngoing(true);
demozone.setCurrentDisaster(disaster.getName());
System.out.println(disaster);

demoRescue.action(0, null, demozone, disaster, null, null, tm);
demoAmbulance.action(0,null,demozone,disaster,null,sm,tm);
demoPolice.action(0, null, demozone, disaster, null, null, tm);

System.out.println("\nDaily attributes Before End OF The Day");
System.out.printf("\n===== DAILY SUMMARY FOR DEMO DAY =====\n");
System.out.printf("%-10s | %-10s | %-10s | %-10s | %-10s | %-16s | %-20s\n",
    "Disasters","Births", "Deaths", "Injuries", "Rescues", "Total Population", "Food in the city (%)");
System.out.println("-----");
System.out.printf("%-10d | %-10d | %-10d | %-10d | %-10d | %-16d | %-20d\n",
    Event.getDailyDisasters(), Event.getDailyBirths(), Event.getDailyDeaths(), Event.getDailyInjured(),
    Event.getDailyrescued(), Event.getTotalPopulation(), Event.getTotalFoodPercentage());
System.out.println("-----");

IEnd.EndOfTheDayEvent();
System.out.println("\nDaily attributes After End OF The Day");
System.out.printf("\n===== DAILY SUMMARY FOR DEMO DAY =====\n");
System.out.printf("%-10s | %-10s | %-10s | %-10s | %-10s | %-16s | %-20s\n",
    "Disasters","Births", "Deaths", "Injuries", "Rescues", "Total Population", "Food in the city (%)");
System.out.println("-----");
System.out.printf("%-10d | %-10d | %-10d | %-10d | %-10d | %-16d | %-20d\n",
    Event.getDailyDisasters(), Event.getDailyBirths(), Event.getDailyDeaths(), Event.getDailyInjured(),
    Event.getDailyrescued(), Event.getTotalPopulation(), Event.getTotalFoodPercentage());
System.out.println("-----");
Event.resetTotalValues();
```


Explanation: The demo clearly shows how the system uses subtyping and multityping. The demoevent object (an EndOfTheDayEvent) is accessed through the abstract class Event and two different interfaces — EndOfTheDay and WeatherObserver. This proves that the same object can be treated as multiple types, each exposing different behaviours such as resetting daily values or responding to weather changes.

Similarly, the demoDisaster object (a FloodDisaster) is accessed both as the abstract class Disaster and the interface SeverityCalculator. This shows how one object can serve as a general disaster in the simulation while also providing its specific severity-calculation logic.

4.7 Polymorphism

4.7.1 Polymorphism by Inclusion

Definition: Inclusion polymorphism (also known as subtype polymorphism) is a form of polymorphism that occurs when a subclass object can be treated as an instance of its superclass or interface.

How I applied: In my project, inclusion polymorphism is effectively demonstrated in several ways. All specific disaster types (like floods, earthquakes, or fires) are handled through a common Disaster reference, allowing uniform processing of different disaster objects. Similarly, various event types are seamlessly scheduled and managed through a single EventCalendar, showcasing polymorphic behavior at the event level. Additionally, polymorphism is applied when passing specialized units such as RescueUnit or VolunteerUnit in place of the more general ResourceUnit within the RescueArrivalEvent, enabling flexible and interchangeable use of related object types.

Where it appears: DisasterOccurEvent, EventCalendar, Resource unit parameter in RescueOperationEndEvent, Resource unit parameter in AmbulanceOperationEndEvent and

Code Snippets:

```
int hour = this.getHour();
for (int i = 0; i < numberOfDisasters ; i++) {

    Zone zone = availableZones.get(i);
    Disaster disaster = randomDisaster(rand);
    int severity = disaster.determineSeverity(Event.getLatestWeather());

    calendar.scheduleEvent(new SingleDisasterEvent(hour, disaster,zone, severity,sm, calendar,tm));
    hour+=3;
}

private Disaster randomDisaster(Random rand) {
    List<Disaster> disasters = sm.getDisasters();
    return disasters.get(rand.nextInt(disasters.size()));
}
```

Explanation: Here, although the variable disaster is of type Disaster, it can hold any specific disaster type (like FloodDisaster, EarthquakeDisaster FireAccidentDisaster, etc.). When the determineSeverity() method is called, Java automatically invokes the overridden method in the actual subclass (for example, FloodDisaster.determineSeverity()).

```

while (calendar.hasNext()) {
    Event e = calendar.nextEvent();
    ws.register(e);
    tm.advanceTime(e.getHour() - tm.getTotalHours());
    e.process();
}

```

Explanation: This is from the Main class where all the scheduled events are processed. Here, Event e holds all types of events and when e.process() called it invokes the method of the actual event.

```

public RescueOperationEndEvent(int hour, Zone zone, Disaster disaster, Resource unit, SimulationManager sm, TimeManager tm) {
    super(hour, 4);
    this.zone = zone;
    this.disaster = disaster;
    this.unit = unit;
    this.sm = sm;
    this.tm = tm;
}

```

```

if (unit != null) {
    // Rescue team is available
    System.out.println(tm.getFormattedTime()+" Rescue Team- "+unit.getId()+" arrived and rescuing "+D.getTrapped()+
        " people in " + zone.getName());
    );
    calendar.scheduleEvent(new RescueOperationEndEvent(this.getHour()+1, zone, D, unit, sm, tm));
} else {
    // No rescue team available - volunteer team steps in
    int v = volunteerUnits.size()+1;
    this.vUnit = new VolunteerTeam("Volunteer "+v);
    sm.addVolunteerTeam(vUnit);
    vUnit.setAvailable(false);
    System.out.println(tm.getFormattedTime()+" Volunteer Team- "+vUnit.getId()+" arrived and rescuing "+D.getTrapped()+
        " people in " + zone.getName());
    calendar.scheduleEvent(new RescueOperationEndEvent(this.getHour()+1, zone, D, vUnit, sm, tm));
}

```

Explanation: Here we are passing Rescue unit (unit) and Volunteer Unit (vUnit) in the place of Resource.

4.7.2 Parametric Polymorphism

Definition: Parametric polymorphism allows code — such as classes or methods — to operate on objects of various types using type parameters, without being limited to a specific data type. This is achieved through Java Generics, enabling type-safe and reusable components that work with multiple data types while maintaining compile-time checking. Simply use of generics to operate on objects of various types.

How I applied: In the SimulationManager class, parametric polymorphism is implemented through generic classes like EntityRegistry<T> and ResourceRegistry<T>.

Each registry stores and manages different types of entities — such as Hospital, Zone, RescueUnit, and VolunteerTeam — using the same reusable, type-safe structure.

Where it appears: generic classes EntityRegistry<T extends Entity> and ResourceRegistry<T extends Resource> and usage in SimulationManager.

Code Snippets:

```

public class EntityRegistry<T extends Entity> {
    private List<T> entities = new ArrayList<>();

    public void add(T entity) {
        entities.add(entity);
    }

    public void remove(T entity) {
        entities.remove(entity);
    }

    public List<T> getAll() {
        return entities;
    }
}

```

```

public class ResourceRegistry<T extends Resource> {
    private List<T> resources = new ArrayList<>();

    public void add(T resource) {
        resources.add(resource);
    }

    public void remove(T resource) {
        resources.remove(resource);
    }

    public List<T> getAll() {
        return resources;
    }
}

```

Explanation: Here, the same EntityRegistry and ResourceRegistry classes are used with different type parameters (T), allowing the system to handle various resources and entities without rewriting code for each type.

4.7.3 Coercion Polymorphism

Definition: type conversion or casting.

How I applied: I used coercion by explicitly casting an object of type Resource to the object of type Rescue unit or Volunteer Team.

Where it appears: Downcasting Resource to VolunteerTeam.

Code Snippet:

```

unit.action(0,this,zone, disaster, null, sm, tm);
unit.setAvailable(true);
if (unit instanceof VolunteerTeam) {
    sm.removeVolunteerTeam((VolunteerTeam) unit);
    unit = null;
}

```

Explanation: Here Volunteer Team is built temporarily, after its use we must remove it from our list. So, downcasting is used to access subclass-specific methods. I did this show explicitly how coercion works.

4.7.4 Overloading

Definition: Multiple methods with the same name but different parameters.

How I applied: In my project Overloading applied by creating two constructors of Hospital with different parameters.

Where it appears: Hospital and Disasters

Code Snippets:

```
public Hospital(String name, int capacity, Zone zone, SimulationManager sm) {
    super(name);
    this.capacity = capacity;
    this.zone = zone;
    this.patientsRemaining = 0;
    this.patientsDischarged = 0;
    this.dailyBirths = 0;
    this.dailyDeaths = 0;
    sm.addHospital(this);
    if (zone != null) {
        zone.setHospital(this);
    }
}

public Hospital(Zone zone, SimulationManager sm) {
    super("No name");
    this.capacity = 100;
    this.zone = zone;
    this.patientsRemaining = 0;
    this.patientsDischarged = 0;
    this.dailyBirths = 0;
    this.dailyDeaths = 0;
    sm.addHospital(this);
    if (zone != null) {
        zone.setHospital(this);
    }
}
```

Explanation: Here the same constructor method can be called with and without name and capacity.

```
public FloodDisaster(int severity, Zone zone) {
    super("Flood", severity, zone);
}

public FloodDisaster() {
    super("Flood");
}
```

Explanation: Here the same constructor method FloodDisaster can be called with and without parameters. This is also same for all the disasters.

4.8 Exceptions

Definition: An exception is an unexpected event or error that occurs during program execution and disrupts the normal flow of instructions.

Where it appears: Custom Exceptions `SimulationException` and `FoodShortageException`.

Code Snippet:

```
public class SimulationException extends Exception {
    public SimulationException(String message) {
        super(message);
    }
}
```

```
public class FoodShortageException extends Exception {
    public FoodShortageException(String message) {
        super(message);
    }
}
```

Explanation: I implemented custom checked exceptions — `SimulationException` and `FoodShortageException` — to provide structured and meaningful error handling within the simulation. `FoodShortageException` is used to manage critical situations like a drop in global food supply below 0, while `SimulationException` handles broader simulation-related issues such as hospital overcapacity, population inconsistencies, or other unexpected conditions. This design ensures that the simulation reacts gracefully to exceptional scenarios instead of abruptly terminating, maintaining realism and system stability.

The above exceptions are explicitly thrown in the methods responsible for modifying Food percentage, total population and etc.

```
public static void minusTotalFoodPercentage(int num) throws FoodShortageException {
    Event.totalFoodPercentage -= num;
    if (totalFoodPercentage < 0) {
        totalFoodPercentage = 0;
        throw new FoodShortageException("⚠ Food supply became 0%! Emergency situation detected.");
    }
}
```

```
public void admitPatients(int number) throws SimulationException {
    this.patientsRemaining += number;
    this.totalPatientsAdmitted += number;
    this.dailyAdmits += number;
    if(this.getAvailablePlaces()<=0) {
        throw new SimulationException("Hospital: " + this.getName()
            + " at Zone: "+this.getZone().getName()+ " capacity exceeded!");
    }
}
```

```

public static void minustotalPopulation(int num) throws SimulationException {
    Event.totalPopulation -= num;
    if (Event.getTotalPopulation() <= 0 ) {
        throw new SimulationException(
            "Total population became 0. Resetting the total population" );
    }
}

```

4.9 Exception Handling

Definition: Exception handling is a mechanism in object-oriented programming that allows a program to detect, manage, and recover from unexpected runtime errors without crashing. It ensures the smooth flow of execution even when abnormal situations occur, by using constructs like try, catch, and throw.

Where it appears: Single Disaster Event to decrease food percentage, Disaster to decrease the dead population and etc.

Code Snippet:

```

try {
    Event.minusTotalFoodPercentage(num);
    System.out.println(tm.getFormattedTime()+" Total Food in the city reduced by " +
        num + "%". Current: " + Event.getTotalFoodPercentage() + "%");
} catch (FoodShortageException e) {
    System.out.println(e.getMessage());
    System.out.println("Emergency food aid dispatched");
    calendar.scheduleEvent(new FoodSupplyEvent(this.getHour()+1,sm,tm));
}

```

```

try {
    Event.minustotalPopulation(dead);
} catch (SimulationException e) {
    System.out.println(e.getMessage());
    Event.setTotalPopulation(800);
}

```

```

try {
    admitPatients(injured,zone,sm,tm);
} catch (SimulationException e) {
    System.out.println(tm.getFormattedTime() + " ⚠ " + e.getMessage());
}

```

Explanation: In the first code we catch the exception if food percentage becomes below 0 and immediately we call the food supply event. Then if the total population becomes 0 we will reset it to 800. If the hospital's capacity is full then we display the Exception message.

5 Simulation Logic

The simulation in this project models how a city responds dynamically to multiple types of disasters over time. It is designed using an **event-driven simulation** approach, where different types of events — such as disasters, rescues, hospital transfers, weather changes, and food supply operations — are scheduled and processed sequentially through an **Event Calendar**.

At the core of the simulation is the **EventCalendar**, which acts as a scheduler and priority queue(priority depends on the scheduled time and priority number) for all upcoming events. Each event (e.g., SingleDisasterEvent, RescueArrivalEvent, HospitalTreatmentEvent, EndOfTheDayEvent) is a subclass of the abstract Event class. The Event class defines the basic structure of all events and declares the abstract method process(), which each specific event overrides to define its unique behavior.

The **TimeManager** keeps track of the simulation's current time and advances the clock as each event is executed. The **WeatherStation** generates and updates weather conditions, which directly influence the severity and frequency of disasters through the **observer mechanism**.

The **SimulationManager** maintains all entities involved in the simulation, such as hospitals, zones, and response units (rescue teams, ambulances, police, and volunteers). These are stored and managed through generic registries, allowing flexible access and reuse across different parts of the program.

When the simulation starts, it runs in a loop for a set number of hours (e.g., 96 hours = 4 simulated days). At specific time intervals:

- **Disaster events** are triggered in random zones using DisasterOccurEvent, which selects a random registered disaster type from the Disaster class.
- Once a disaster occurs, the affected zone generates casualties (injured, deaths, and trapped people) based on the disaster's severity.
- **Response units** such as rescue teams and ambulances are dispatched to handle rescue operations and hospital transfers.
- **HospitalTreatmentEvent** processes the treatment of admitted patients periodically.
- **EndOfTheDayEvent** summarizes daily statistics (deaths, rescues, births, food status, etc.) and resets counters for the next day.
- **WeatherChangeEvent** changes the weather periodically, influencing upcoming disaster severities.

All major entities (zones, hospitals, units, and resources) interact through message passing and event scheduling rather than direct coupling, ensuring modular and extensible system design.

At the end of the simulation, a **final summary** is generated that displays the total number of disasters, deaths, rescues, injuries, and remaining city resources such as population and food percentage. This provides a complete overview of how effectively the city responded to all the disasters during the simulation period.

Main class logic:

```

int totalHours = 96;
for (int hour = 0; hour < totalHours; hour++) {
    int hourOfDay = hour % 24;
    if (hour % 6 == 0 && hour != 0)
        calendar.scheduleEvent(new WeatherChangeEvent(hour,ws,tm));
    if (hourOfDay == 6) {
        calendar.scheduleEvent(new DisasterOccurEvent(hour,sm,calendar,tm));
    }

    if (hour % 10 == 0 && hour != 0) {
        calendar.scheduleEvent(new HospitalTreatmentEvent( hour, sm,  tm));
    }
    if (hour % 24 == 23) calendar.scheduleEvent(new EndOfTheDayEvent( hour, sm, tm));

}
int lastDay = 0;

while (calendar.hasNext()) {
    Event e = calendar.nextEvent();
    ws.register(e);
    tm.advanceTime(e.getHour() - tm.getTotalHours());
    e.process();
}

```

Event Calendar class:

```

public class EventCalendar {
    private PriorityQueue<Event> Queue;
    public EventCalendar() {
        this.Queue = new PriorityQueue<>();
    }

    public void scheduleEvent(Event event) {
        Queue.add(event);
    }

    public Event nextEvent() {
        return Queue.poll();
    }

    public boolean hasNext() {
        return !Queue.isEmpty();
    }
}

```

Time Manager class:


```

public class TimeManager {
    private int totalHours = 0;
    public void advanceTime(int hours) {
        totalHours += hours;
    }
    public int getCurrentDay() {
        return (totalHours / 24) + 1;
    }
    public int getCurrentHour() {
        return totalHours % 24;
    }
    public int getTotalHours() {
        return totalHours;
    }
    public String getFormattedTime() {
        return "[Day:" + getCurrentDay() + " - Hour:" + getCurrentHour()+"]";
    }
}

```

6. Extensibility of project

The project is designed with a high level of **extensibility**, meaning new features, components, and behaviors can be added without modifying the existing codebase. This is achieved through the use of **abstract classes**, **interfaces**, and **clearly separated responsibilities**.

- For example the abstract class Disaster act as flexible blueprints. New disasters (e.g., *Tornado*, *Wildfire*) can be added simply by extending these classes and implementing their abstract methods and determines severity method which is from the interface SeverityCalculator.
- Interfaces like EndOfTheDay, WeatherObserver, and SeverityCalculator allow new classes to adopt additional behaviors independently. For example, any new disaster can easily implement SeverityCalculator to provide its own severity logic.
- The system's registry and manager structure ensures that newly created objects can be plugged into the existing simulation without any structural changes.
- New event types can be added simply by extending Event and implementing the process() method, since the EventCalendar treats all events polymorphically. Even new resource units (like a MedicalDroneTeam) can be added by extending Resource, and they will integrate seamlessly into rescue or ambulance workflows.

This architecture avoids rewriting existing logic, reduces coupling, and ensures that the simulation can evolve naturally as new requirements arise—making the system highly future-proof and scalable. Because of this design, future expansion—new types of disasters, new event categories, new weather effects, or new simulation rules—can be implemented with minimal effort and without affecting the stability of the existing system. This makes the project **scalable, modular, and easy to maintain**.

7. Conclusion

In summary, the Disaster Management Simulation Project effectively demonstrates the practical implementation of fundamental Object-Oriented Programming (OOP) principles, including abstraction, inheritance, polymorphism, encapsulation, composition, and exception handling. The simulation

models a realistic and dynamic environment where multiple disasters occur, resources are deployed, hospitals respond, and events progress in a time-driven sequence. The modular and extensible design ensures that new disaster types, event categories, or resource units can be easily incorporated with minimal modification to existing code, emphasizing the system's scalability and maintainability. Furthermore, the project integrates robust exception handling to ensure reliability under extreme simulation conditions, such as hospital overcapacity or resource shortages. Overall, this project not only fulfills its technical objectives but also illustrates how OOP concepts can be effectively applied to simulate and manage complex real-world disaster response scenarios.